I. E. S.
Infanta Elena
Jumilla



UT 2
Introducción a Android

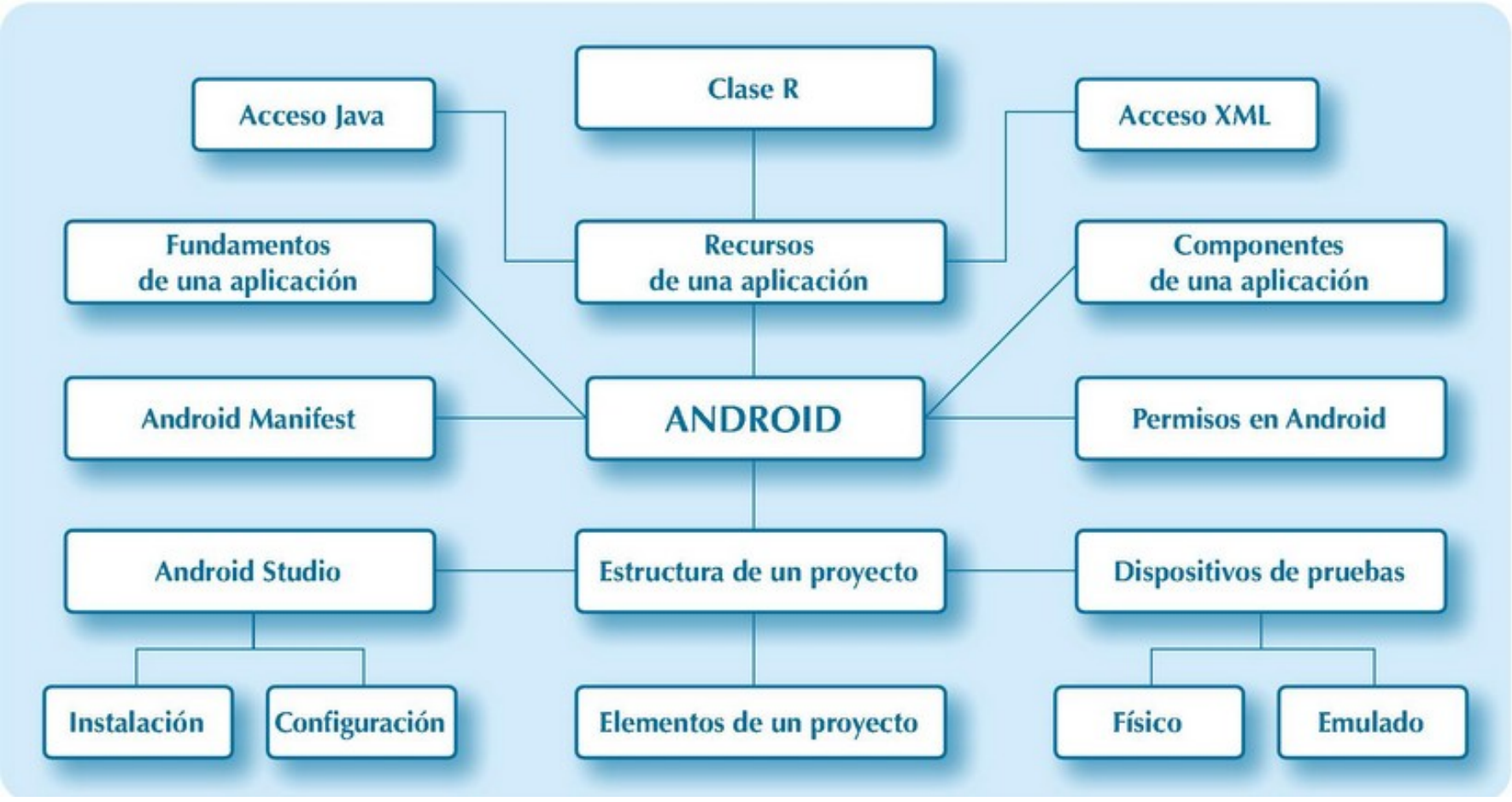
2º DAM - PMDM

Profesor: **Daniel Vicente Fernández**

Índice

- Definición
- Arquitectura del SO Android
- Ciclo de vida de una aplicación
- Fundamentos de una aplicación
- Componentes de una aplicación
- *AndroidManifest*
- Recursos de una aplicación
- La clase R
- Estructura de un proyecto Android
- Los permisos en Android

Mapa conceptual



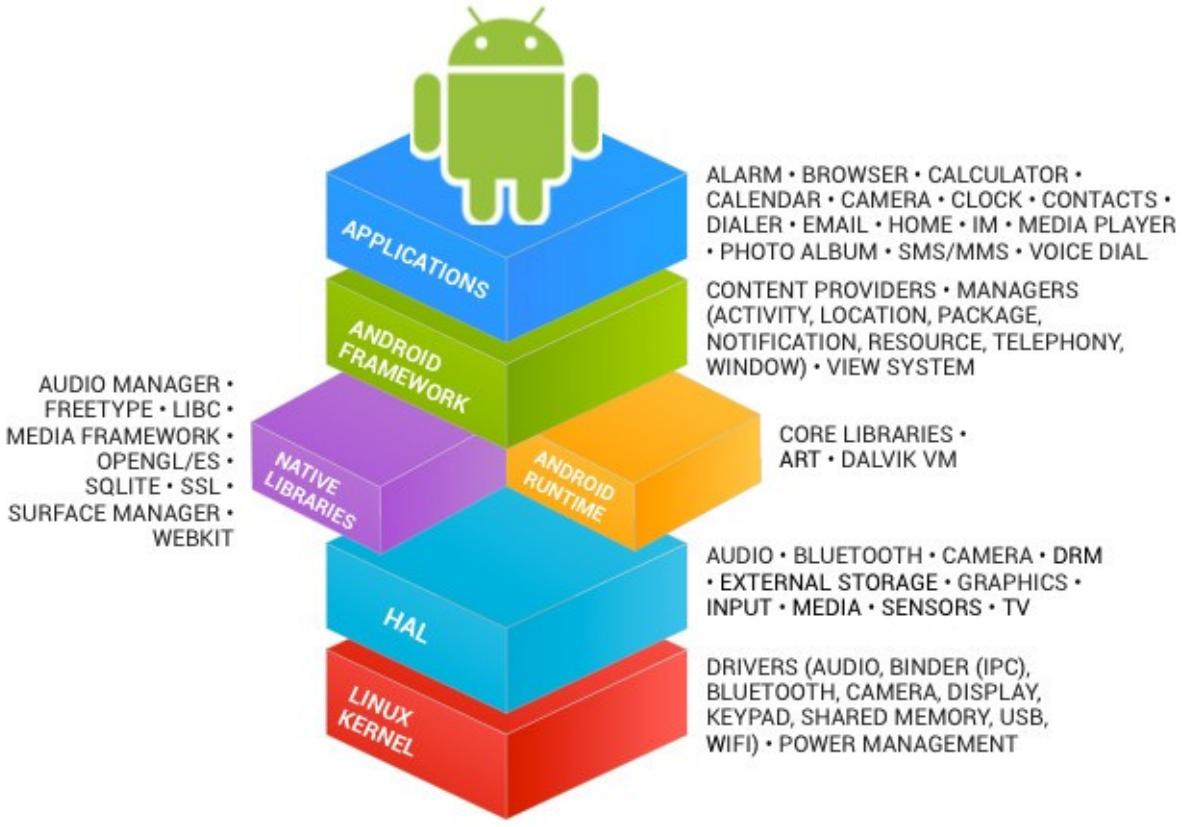
Glosario

- **Aplicaciones responsivas**. Son aquellas que se adaptan a diferentes tamaños y proporciones de pantalla mediante un cambio o reorganizando del contenido en el interfaz, que puede suponer el escalado de imágenes, cambio de orden o posición en los menús.
- ***Broadcast***. En Android, la difusión amplia, difusión ancha o *broadcast* es la transmisión de datos que serán recibidos por todos los elementos que se encuentren activos y operativos en el dispositivo móvil.
- ***Bundle***. Son sistemas de almacenaje (paquetes) que se usan para pasar información entre diferentes actividades de Android.
- ***Grandle Scripts***. Carpeta de un proyecto de Android que contiene información necesaria para la compilación del proyecto.
- ***Intent***. Importante elemento de Android que es utilizado para cambiar de una pantalla o actividad a otra.
- **Kit de desarrollo de software (*SDK*)**. Se denomina así al conjunto de herramientas de desarrollo de software que permite al programador crear una aplicación informática para un sistema concreto.
- ***Platform***. Término que se aplica generalmente en sistemas operativos a una o varias tecnologías que se utilizan como base sobre la que desarrollar aplicaciones, establecer procesos o implementar nuevas tecnologías.

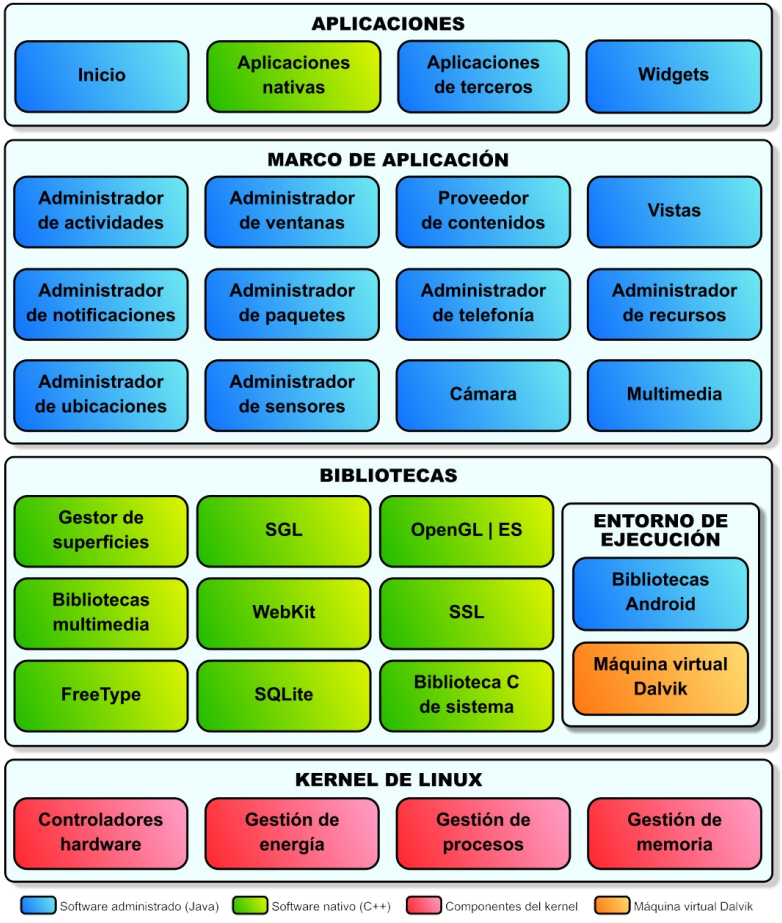
Definición

- Android es el sistema operativo móvil más popular y utilizado en todo el mundo.
- Desarrollado por **Google**.
- Multiusuario y multitarea basado en el núcleo de **Linux**.
- Diseñado principalmente para dispositivos móviles con pantallas táctiles, como smartphones y tablets.
- Lenguajes de programación habituales: **Java** y **Kotlin**.
- Las aplicaciones se distribuyen principalmente a través de **Google Play Store**, lo que facilita el acceso a un mercado global.

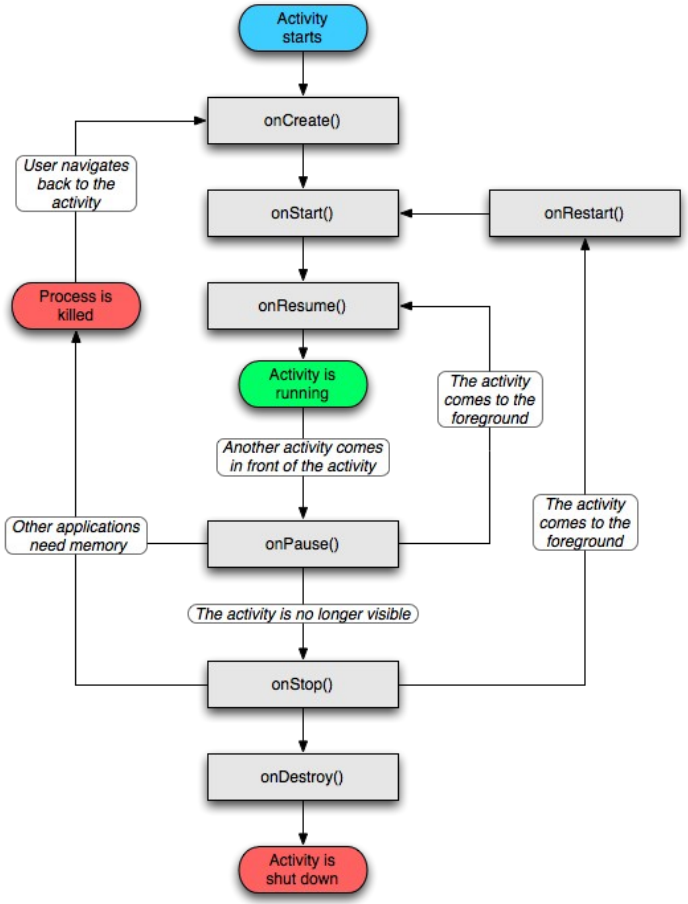
Arquitectura del SO Android



Evolución de Android

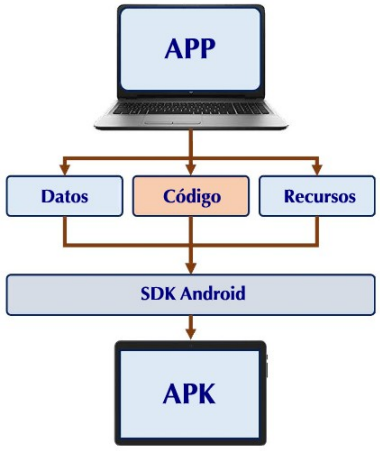


Ciclo de vida de una aplicación



Fundamentos de una aplicación

- Cada aplicación tiene un propietario único y diferente (*id_user*).
  - Cada aplicación está aislada del resto: permisos para cada usuario, procesos propios, máquinas virtuales propias (*Dalvik/ART*), etc.
- **Principio de menor privilegio**: cada aplicación tiene los privilegios justos y necesarios.
- Cada aplicación debe solicitar permiso para acceder a otros recursos: cámara, GPS, agenda, etc.
- Una aplicación esta escrita en **Java** o **Kotlin** que son lenguajes de programación orientados a objetos.
- El SDK (Kit de Desarrollo Software) compila datos, código y recursos en un único fichero APK (*Android **A**pplication **P**ac**k**age*)



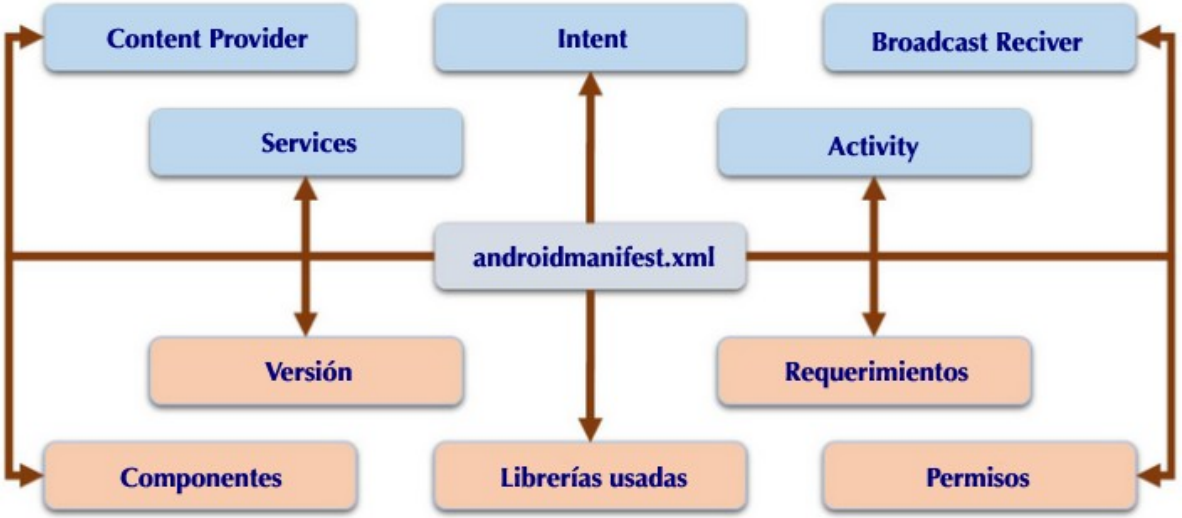
Componentes de una aplicación

- **Componente**: bloque de código de una aplicación mediante los cuales el sistema puede interaccionar con ella.
  - ***Activity***: interfaz visual de una aplicación. Cada pantalla de la aplicación es una *Activity*, que se maneja en un ciclo de vida específico controlado por el sistema.
  - ***Service***: realiza operaciones en segundo plano sin una UI. Se utiliza para tareas prolongadas como la reproducción de música o la sincronización de datos. Las *activities* pueden iniciar un servicio o enlazarse con él.
  - ***Broadcast Receiver***: permite que la aplicación escuche mensajes o eventos del sistema, como cuando el dispositivo termina de cargarse o recibe una llamada.
  - ***Content Provider***: facilita el intercambio de datos entre aplicaciones. Por ejemplo la *app* de contactos ofrece acceso a los contactos almacenados para otras aplicaciones.
- Cada componente tiene una función específica y su propio ciclo de vida diferente.
- Una aplicación Android está formada por uno o más componentes.

Android Manifest

- ***AndroidManifest.xml***: elemento clave en cualquier aplicación Android.
- Define información esencial sobre la aplicación, como:
  - El **nombre del paquete** (que debe ser único).
  - Los **permisos** que necesita la aplicación para funcionar (como acceder a Internet o leer el almacenamiento). *`uses-permission`*
  - Las **actividades**, **servicios**, y otros **componentes** que incluye la app.
  - La **versión mínima del SDK** de Android requerido. *`uses-sdk`*
- Este archivo le dice al sistema operativo cómo debe ejecutarse la aplicación y qué recursos o características necesita.

Android Manifest II




```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.PrimeraAplicacion"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="@string/app_name"
            android:theme="@style/Theme.PrimeraAplicacion">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Recursos de una aplicación

- Datos que utiliza una aplicación: imágenes, texto, estilos, carpetas, etc.
- Pueden estar predeterminados para cada aplicación o crearse cuando se necesiten según la configuración del dispositivo.
- Suelen ser gestionados a través de ficheros XML por lo que deberán tener un identificador único (id).
- Los archivos de recursos deben almacenarse en alguna subcarpeta dentro de la carpeta *res*.

Carpetas de recursos de una aplicación

- `res/`**`drawable`**: recursos dibujables (bitmaps, imágenes escalables, formas, colores, etc.).
- `res/`**`layout`**: ficheros XML que definen los elementos visuales de la interfaz de la aplicación.
- `res/`**`animator`**: ficheros XML que definen animaciones sencillas sobre elementos gráficos (rotaciones, *fading*, movimiento y estiramiento).
- `res/`**`mipmap`**: similar a drawable pero para bitmaps como el icono de la aplicación.

Carpetas de recursos de una aplicación II

- `res/`**`menu`**: ficheros XML que definen las diferentes opciones de menús, barras de navegación, etc.
- `res/`**`values`**: ficheros XML que definen aspectos generales de la aplicación (color, dimensión, cadenas de texto, estilos, etc.).
- `res/`**`xml`**: ficheros XML que utiliza Android para dar soporte a treas múltiples. Por ejemplo, configurar búsquedas.
- `res/`**`raw`**: almacena los archivos RAW de la aplicación (audio y vídeo). También se puede utilizar la carpeta `asset` de Android si es necesario respetar el nombre real del fichero.

Más información importante

Acceso a los recursos

- Android genera automáticamente un ID para cada recurso.
- En la **clase R** de la aplicación queda registrado la relación entre el ID y el tipo de recurso.
- Esto permite acceder al recurso desde XML, Java o Kotlin.

Acceso a los recursos en XML

Sintaxis: *@[<package_name>:]<resource_type>/<resource_name>*

- ***<package_name>*** es el nombre del paquete en el que se ubica el recurso (no es necesario cuando se hace referencia a recursos desde el mismo paquete).
- ***<resource_type>*** es la subclase R para el tipo de recurso.
- ***<resource_name>*** es el nombre de archivo del recurso sin la extensión ni el valor del atributo android:name en el elemento XML (para valores simples).

```
<!-- Archivo de recurso: -->
<?xml version="1.0" encoding="utf-8"?>
<resources>
   <color name="opaque_red">#f00</color>
   <string name="hello">Hello!</string>
</resources>

<!-- Archivo de diseño: -->
<?xml version="1.0" encoding="utf-8"?>
<EditText xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:textColor="@color/opaque_red"
    android:text="@string/hello"
/>
```

Acceso a los recursos en código

Sintaxis: *[<package_name>.]R.<resource_type>.<resource_name>*

- ***<package_name>*** es el nombre del paquete en el que se ubica el recurso (no es necesario cuando haces referencia a recursos desde tu paquete).
- ***<resource_type>*** es la subclase R para el tipo de recurso.
- ***<resource_name>*** es el nombre de archivo del recurso sin la extensión ni el valor del atributo android:name en el elemento XML (para valores simples).

```
// Carga un color de fondo para la pantalla actual desde un recurso dibujable (drawable)
window.setBackgroundDrawableResource(R.drawable.my_background_image)

// Establece el título del Activity obteniendo la cadena desde el objeto Resources.
// Ya que este método necesita un CharSequence y no un ID
window.setTitle(resources.getText(R.string.main_title))

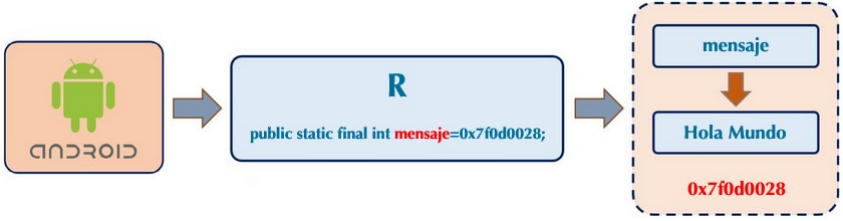
// Carga un layout personalizado para la pantalla actual
setContentView(R.layout.main_screen)

// Establece una escena en la animación obteniendola desde el objeto Resources
flipper.setInAnimation(AnimationUtils.loadAnimation(this, R.anim.hyperspace_in))

// Establece el texto en un objeto TextView a partir del ID del recurso
val msgTextView = findViewById(R.id.msg) as TextView
msgTextView.setText(R.string.hello message)
```

La clase R en Android

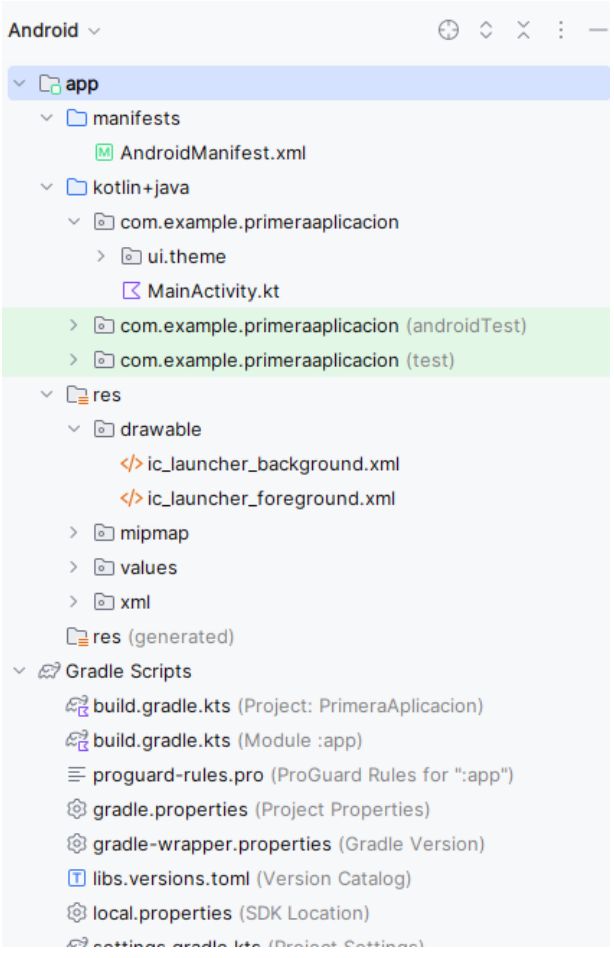
- Contiene variables estáticas que identifican cada tipo de recurso.
- Android lee el fichero XML, carga todas las estructuras solicitadas en memoria y mantiene el fichero R como referencia directa a los recursos cargados.
- El archivo R.java lo crea automáticamente a través de la herramienta AAPT (*Android Asset Packaging Tool*) cuando se compila el proyecto.
- Por lo tanto si se modifica a mano se perderán los coambios tras la compilación



mensaje contiene la cadena “Hola Mundo“ que está almacenado en una dirección de memoria concreta

Estructura de un proyecto Android

- app: carpeta principal del proyecto
  - `manifest`: AndroidManifest.xml
  - `kotlin+java`: código fuente
  - `res`: ficheros de recursos
- `Gradle Scripts`: contiene información necesaria para la compilación del proyecto (versión de SDK, versión mínima soportada de Android, referencias a librerías, etc.)



Permisos en Android

A partir de Android 6.0 (*Marshmallow*) los permisos se pueden conceder y revocar en cualquier momento.

El programador debe:

1) Declarar en *AndroidManifest* los permisos más críticos que requiere la aplicación.
2) La aplicación debe comprobar si tiene concedido el permiso cada vez que lo necesite y solicitarlo en caso negativo.

```
int permissionCheck=ContextCompat.CheckSelfPermission(thisActivity, Manifest.permission.CAMERA);
```

¡¡Fin!!